Collapse Sidebar

Section Navigation

sklearn
sklearn.base
sklearn.calibration
sklearn.callback
sklearn.cluster
sklearn.compose
sklearn.covariance
sklearn.cross_decomposition
sklearn.datasets
sklearn.decomposition
sklearn.discriminant_analysis
sklearn.dummy
sklearn.ensemble
sklearn.exceptions
sklearn.experimental
sklearn.feature_extraction
sklearn.feature_selection
sklearn.frozen
sklearn.gaussian_process
sklearn.impute
IterativeImputer
KNNImputer
MissingIndicator
SimpleImputer
sklearn.inspection
sklearn.isotonic
sklearn.kernel_approximation
sklearn.kernel_ridge
sklearn.linear_model
sklearn.manifold
sklearn.metrics
sklearn.mixture
sklearn.model_selection
sklearn.multiclass
sklearn.multioutput
sklearn.naive_bayes
sklearn.neighbors
sklearn.neural_network
sklearn.pipeline
sklearn.preprocessing
sklearn.random_projection
sklearn.semi_supervised
sklearn.svm
sklearn.tree
sklearn.utils

IterativeImputer

class sklearn.impute.IterativeImputer(*estimator=None, *, missing_values=nan, sample_posterior=False, max_iter=10, tol=0.001, n_nearest_features=None, initial_strategy='mean', fill_value=None, imputation_order='ascending', skip_complete=False, min_value=-inf, max_value=inf, verbose=0, random_state=None, add_indicator=False, keep_empty_features=False) [source]

Multivariate imputer that estimates each feature from all the others.

A strategy for imputing missing values by modeling each feature with missing values as a function of other features in a round-robin fashion.

Read more in the User Guide.

Added in version 0.21.

Note

This estimator is still **experimental** for now: the predictions and the API might change without any deprecation cycle. To use it, you need to explicitly import `enable_iterative_imputer`:

```
>>> # explicitly require this experimental feature
>>> from sklearn.experimental import enable_iterative_imputer  # noqa
>>> # now you can import normally from sklearn.impute
>>> from sklearn.impute import IterativeImputer
```

Parameters:

estimator : *estimator object, default=BayesianRidge()*

The estimator to use at each step of the round-robin imputation. If `sample_posterior=True`, the estimator must support `return_std` in its `predict` method.

missing_values : *int or np.nan, default=np.nan*

The placeholder for the missing values. All occurrences of `missing_values` will be imputed. For pandas' dataframes with nullable integer dtypes with missing values, `missing_values` should be set to `np.nan`, since `pd.NA` will be converted to `np.nan`.

sample_posterior : *bool, default=False*

Whether to sample from the (Gaussian) predictive posterior of the fitted estimator for each imputation. Estimator must support `return_std` in its `predict` method if set to `True`. Set to `True` if using `IterativeImputer` for multiple imputations.

max_iter : *int, default=10*

Maximum number of imputation rounds to perform before returning the imputations computed during the final round. A round is a single imputation of each feature with missing values. The stopping criterion is met once `max(abs(X_t - X_{t-1}))/max(abs(X[known_vals])) < tol`, where `X_t` is `X` at iteration `t`. Note that early stopping is only applied if `sample_posterior=False`.

tol : *float, default=1e-3*

Tolerance of the stopping condition.

n_nearest_features : *int, default=None*

Number of other features to use to estimate the missing values of each feature column. Nearness between features is measured using the absolute correlation coefficient between each feature pair (after initial imputation). To ensure coverage of features throughout the imputation process, the neighbor features are not necessarily nearest, but are drawn with probability proportional to correlation for each imputed target feature. Can provide significant speed-up when the number of features is huge. If `None`, all features will be used.

initial_strategy : *{'mean', 'median', 'most_frequent', 'constant'}, default='mean'*

Which strategy to use to initialize the missing values. Same as the `strategy` parameter in `SimpleImputer`.

fill_value : *str or numerical value, default=None*

When `strategy="constant"`, `fill_value` is used to replace all occurrences of missing_values. For string or object data types, `fill_value` must be a string. If `None`, `fill_value` will be 0 when imputing numerical data and "missing_value" for strings or object data types.

Added in version 1.3.

imputation_order : *{'ascending', 'descending', 'roman', 'arabic', 'random'}, default='ascending'*

The order in which the features will be imputed. Possible values:

- `'ascending'`: From features with fewest missing values to most.
- `'descending'`: From features with most missing values to fewest.
- `'roman'`: Left to right.
- `'arabic'`: Right to left.
- `'random'`: A random order for each round.

skip_complete : *bool, default=False*

If `True` then features with missing values during `transform` which did not have any missing values during `fit` will be imputed with the initial imputation method only. Set to `True` if you have many features with no missing values at both `fit` and `transform` time to save compute.

min_value : *float or array-like of shape (n_features,), default=-np.inf*

Minimum possible imputed value. Broadcast to shape `(n_features,)` if scalar. If array-like, expects shape `(n_features,)`, one min value for each feature. The default is `-np.inf`.

Changed in version 0.23: Added support for array-like.

max_value : *float or array-like of shape (n_features,), default=np.inf*

Maximum possible imputed value. Broadcast to shape `(n_features,)` if scalar. If array-like, expects shape `(n_features,)`, one max value for each feature. The default is `np.inf`.

Changed in version 0.23: Added support for array-like.

verbose : *int, default=0*

Verbosity flag, controls the debug messages that are issued as functions are evaluated. The higher, the more verbose. Can be 0, 1, or 2.

random_state : *int, RandomState instance or None, default=None*

The seed of the pseudo random number generator to use. Randomizes selection of estimator features if `n_nearest_features` is not `None`, the `imputation_order` if `random`, and the sampling from posterior if `sample_posterior=True`. Use an integer for determinism. See the Glossary.

add_indicator : *bool, default=False*

If `True`, a `MissingIndicator` transform will stack onto output of the imputer's transform. This allows a predictive estimator to account for missingness despite imputation. If a feature has no missing values at fit/train time, the feature won't appear on the missing indicator even if there are missing values at transform/test time.

keep_empty_features : *bool, default=False*

If True, features that consist exclusively of missing values when `fit` is called are returned in results when `transform` is called. The imputed value is always `0` except when `initial_strategy="constant"` in which case `fill_value` will be used instead.

Added in version 1.2.

Attributes:

initial_imputer_ : *object of type `SimpleImputer`*

Imputer used to initialize the missing values.

imputation_sequence_ : *list of tuples*

Each tuple has `(feat_idx, neighbor_feat_idx, estimator)`, where `feat_idx` is the current feature to be imputed, `neighbor_feat_idx` is the array of other features used to impute the current feature, and `estimator` is the trained estimator used for the imputation. Length is `self.n_features_with_missing_ * self.n_iter_`.

n_iter_ : *int*

Number of iteration rounds that occurred. Will be less than `self.max_iter` if early stopping criterion was reached.

n_features_in_ : *int*

Number of features seen during fit.

Added in version 0.24.

feature_names_in_ : *ndarray of shape (`n_features_in_`,)*

On this page

IterativeImputer
fit
fit_transform
get_feature_names_out
get_metadata_routing
get_params
set_output
set_params
transform

Gallery examples

scikit-learn is financially supported by Probabl and other institutions.

:probabl.

Enterprise-grade solutions and services

Names of features seen during fit. Defined only when `X` has feature names that are all strings.

Added in version 1.0.

n_features_with_missing_ : *int*

Number of features with missing values.

indicator_ : *MissingIndicator*

Indicator used to add binary indicators for missing values. `None` if `add_indicator=False`.

random_state_ : *RandomState instance*

RandomState instance that is generated either from a seed, the random number generator or by `np.random`.

See also

`SimpleImputer`
Univariate imputer for completing missing values with simple strategies.

`KNNImputer`
Multivariate imputer that estimates missing features using nearest samples.

Notes

To support imputation in inductive mode we store each feature's estimator during the `fit` phase, and predict without refitting (in order) during the `transform` phase.

Features which contain all missing values at `fit` are discarded upon `transform`.

Using defaults, the imputer scales in $\mathcal{O}(knp^3\min(n,p))$ where k = `max_iter`, n the number of samples and p the number of features. It thus becomes prohibitively costly when the number of features increases. Setting `n_nearest_features << n_features`, `skip_complete=True` or increasing `tol` can help to reduce its computational cost.

Depending on the nature of missing values, simple imputers can be preferable in a prediction context.

References

[1] Stef van Buuren, Karin Groothuis-Oudshoorn (2011). "mice: Multivariate Imputation by Chained Equations in R". Journal of Statistical Software 45: 1-67.

[2] S. F. Buck, (1960). "A Method of Estimation of Missing Values in Multivariate Data Suitable for use with an Electronic Computer". Journal of the Royal Statistical Society 22(2): 302-306.

Examples

```
>>> import numpy as np
>>> from sklearn.experimental import enable_iterative_imputer
>>> from sklearn.impute import IterativeImputer
>>> imp_mean = IterativeImputer(random_state=0)
>>> imp_mean.fit([[7, 2, 3], [4, np.nan, 6], [10, 5, 9]])
IterativeImputer(random_state=0)
>>> X = [[np.nan, 2, 3], [4, np.nan, 6], [10, np.nan, 9]]
>>> imp_mean.transform(X)
array([[ 6.9584,  2.    ,  3.    ],
       [ 4.    ,  2.6000,  6.    ],
       [10.    ,  4.9999,  9.    ]])
```

For a more detailed example see Imputing missing values before building an estimator or Imputing missing values with variants of IterativeImputer.

fit(*X*, *y=None*, ***fit_params*) [source]

Fit the imputer on `X` and return self.

Parameters:

X : *array-like, shape (n_samples, n_features)*

Input data, where `n_samples` is the number of samples and `n_features` is the number of features.

y : *Ignored*

Not used, present for API consistency by convention.

****fit_params** : *dict*

Parameters routed to the `fit` method of the sub-estimator via the metadata routing API.

Added in version 1.5: Only available if `sklearn.set_config(enable_metadata_routing=True)` is set. See Metadata Routing User Guide for more details.

Returns:

self : *object*

Fitted estimator.

fit_transform(*X*, *y=None*, ***params*) [source]

Fit the imputer on `X` and return the transformed `X`.

Parameters:

X : *array-like, shape (n_samples, n_features)*

Input data, where `n_samples` is the number of samples and `n_features` is the number of features.

y : *Ignored*

Not used, present for API consistency by convention.

****params** : *dict*

Parameters routed to the `fit` method of the sub-estimator via the metadata routing API.

Added in version 1.5: Only available if `sklearn.set_config(enable_metadata_routing=True)` is set. See Metadata Routing User Guide for more details.

Returns:

Xt : *array-like, shape (n_samples, n_features)*

The imputed input data.

get_feature_names_out(*input_features=None*) [source]

Get output feature names for transformation.

Parameters:

input_features : *array-like of str or None, default=None*

Input features.

- If `input_features` is `None`, then `feature_names_in_` is used as feature names in. If `feature_names_in_` is not defined, then the following input feature names are generated: `["x0", "x1", ..., "x(n_features_in_ - 1)"]`.
- If `input_features` is an array-like, then `input_features` must match `feature_names_in_` if `feature_names_in_` is defined.

Returns:

feature_names_out : *ndarray of str objects*

Transformed feature names.

get_metadata_routing() [source]

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

Added in version 1.5.

Returns:

routing : *MetadataRouter*

A `MetadataRouter` encapsulating routing information.

get_params(*deep=True*) [source]

Get parameters for this estimator.

Parameters:

deep : *bool, default=True*

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Returns:

params : *dict*

Parameter names mapped to their values.

set_output(*, transform=None) [source]

Set output container.

Refer to the user guide for more details and Introducing the set_output API for an example on how to use the API.

Parameters:

transform : {"default", "pandas", "polars"}, default=None

Configure output of `transform` and `fit_transform`.

- `"default"`: Default output format of a transformer
- `"pandas"`: DataFrame output
- `"polars"`: Polars output
- `None`: Transform configuration is unchanged

> **Added in version 1.4:** `"polars"` option was added.

Returns:

self : estimator instance

Estimator instance.

set_params(**params) [source]

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as `Pipeline`). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

Parameters:

****params : dict**

Estimator parameters.

Returns:

self : estimator instance

Estimator instance.

transform(X) [source]

Impute all missing values in `X`.

Note that this is stochastic, and that if `random_state` is not fixed, repeated calls, or permuted input, results will differ.

Parameters:

X : array-like of shape (n_samples, n_features)

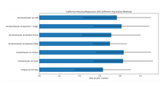
The input data to complete.

Returns:

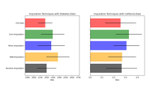
Xt : array-like, shape (n_samples, n_features)

The imputed input data.

Gallery examples



Imputing missing values with variants of IterativeImputer



Imputing missing values before building an estimator

Previous sklearn.impute

Next KNNImputer

© Copyright 2007 - 2026, scikit-learn developers (BSD License).